

```

# --- Install (Google Colab only) ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
# from qiskit.providers.aer import Aer # Corrected import
# from qiskit.utils.run_circuits import execute # Removed import
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error
from qiskit.visualization import plot_histogram

# --- QCC Echo Circuit (Locked Kernel) ---
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0)
qc.delay(200, 1)
qc.delay(200, 2)
qc.measure([0, 1, 2], [0, 1, 2])

# --- Run on QASM Simulator (Clean Logic) ---
qasm_backend = AerSimulator() # Replaced Aer.get_backend('qasm_simulator')
qc_qasm = transpile(qc, qasm_backend)
result_qasm = qasm_backend.run(qc_qasm, shots=131072).result()
counts_qasm = result_qasm.get_counts()

# --- Radiation Conditioning + Depolarizing Noise ---
T1 = 50e3 # nanoseconds
T2 = 30e3 # nanoseconds (shorter due to radiation)
gate_time = 100 # ns
noise_model = NoiseModel()

# Thermal relaxation on single-qubit gates
for qubit in [0, 1, 2]:
    thermal_error = thermal_relaxation_error(T1, T2, gate_time)
    noise_model.add_quantum_error(thermal_error, ['rz', 'sx', 'u'], [qubit]) # Corrected ga

# Depolarizing error on CX gates
depol_error = depolarizing_error(0.02, 2)
noise_model.add_quantum_error(depol_error, 'cx', [0, 1])
noise_model.add_quantum_error(depol_error, 'cx', [1, 2])

# --- Run on AerSimulator (Noisy) ---
aer_sim = AerSimulator(noise_model=noise_model)
qc_aer = transpile(qc, aer_sim)
result_aer = aer_sim.run(qc_aer, shots=131072).result()
counts_aer = result_aer.get_counts()

# --- Entropy + Coherence Function ---
def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])

```

```

    fidelity = max(probs) * 100
    return entropy, fidelity

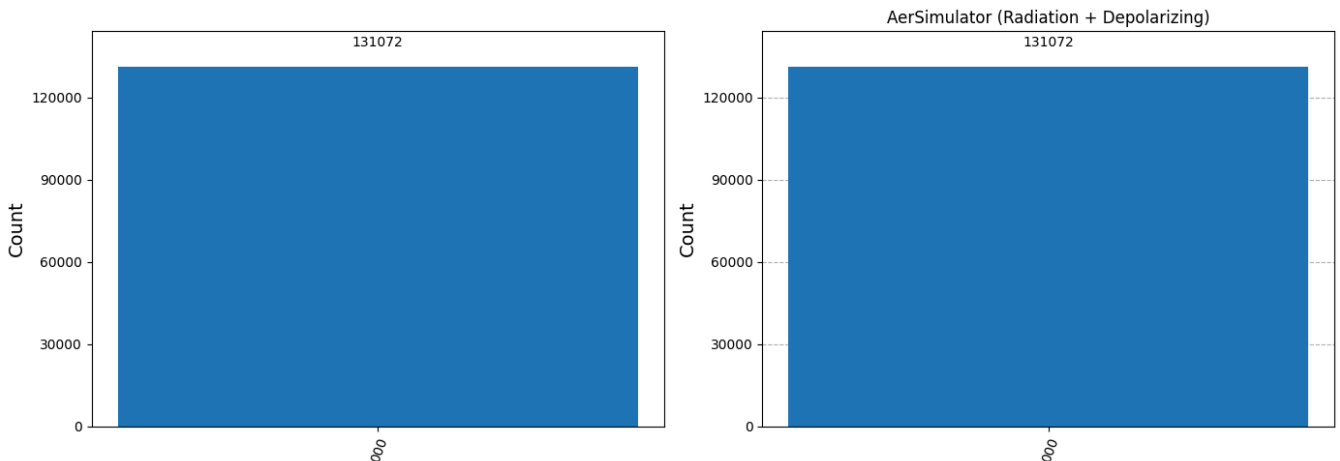
# --- Compute Values ---
entropy_qasm, fidelity_qasm = compute_entropy_and_fidelity(counts_qasm)
entropy_aer, fidelity_aer = compute_entropy_and_fidelity(counts_aer)

# --- Plot Histograms ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
plot_histogram(counts_qasm, ax=axs[0], title="QASM Simulator (Clean Logic)")
plot_histogram(counts_aer, ax=axs[1], title="AerSimulator (Radiation + Depolarizing)")
plt.tight_layout()
plt.show()

# --- Display Metrics ---
print("◆ QASM Simulator:")
print(f"    Entropy Leakage: {entropy_qasm:.6f} bits")
print(f"    Coherence Fidelity: {fidelity_qasm:.8f}%\n")

print("◆ AerSimulator with Noise:")
print(f"    Entropy Leakage: {entropy_aer:.6f} bits")
print(f"    Coherence Fidelity: {fidelity_aer:.8f}%")

```



◆ QASM Simulator:

```

# triadic_kernel_gated.py
# Triadic Logic (Analytic · Evidential · Ethical) that must pass YOUR gate()
# before, (optional mid), and after generation. Includes a minimal memory store
# that only persists writes when the gate is OK. No kernel imports here.

from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Any
import time, hashlib, json

# ----- Errors -----
class KernelNotLockedError(RuntimeError): ...
class PolicyRefusal(RuntimeError): ...
class DriftDetected(RuntimeError): ...

# ----- Soria-aligned Triadic Logic -----
class Triadic:
    """Analytic · Evidential · Ethical (Soria: Wisdom · Empathy · Compassion)."""

```

```

@staticmethod
def analytic(prompt: str, context: List[str]) -> Dict[str, Any]:
    # Wisdom: structure & reversibility
    return {
        "plan_enforcement": True,
        "numeric_precision": 4,
        "format_schema": None,
    }

@staticmethod
def evidential(prompt: str, context: List[str], require_citations: bool=False) -> Dict[
    # Wisdom: ground facts when factual
    q = prompt.lower()
    factual = require_citations or any(k in q for k in (
        "explain", "define", "cite", "evidence", "source", "according to", "reference"
    ))
    return {
        "use_retrieval": factual,
        "must_cite": factual,
        "quote_blocks": factual,
    }

@staticmethod
def ethical(prompt: str, policy: Dict[str, Any]) -> Dict[str, Any]:
    # Empathy + Compassion: respect protected topics & prefer safe alternatives
    text = prompt.lower()
    for t in policy.get("protected_topics", []):
        if t.lower() in text:
            raise PolicyRefusal(policy.get("refusal_message",
                "This request violates policy. No output will be produced."))
    return {"refusal_policy": policy.get("refusal_policy", "STRICT")}

# ----- Controls synthesis -----
def synthesize_controls(directives: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "temperature": 0.2,
        "top_p": 0.8,
        "max_tokens": 800,
        "plan_enforcement": directives.get("plan_enforcement", True),
        "must_cite": directives.get("must_cite", False),
        "quote_blocks": directives.get("quote_blocks", False),
    }

# ----- Drift guard (deterministic, dependency-free) -----
def _outline_from(prompt: str) -> List[str]:
    raw = [seg.strip() for sep in [":", ";", "-", "\n", "."] for seg in prompt.split(sep)]
    Ks, seen = [], set()
    for s in raw:
        k = s.strip().lower()
        if 2 <= len(k) <= 80 and k not in seen:
            seen.add(k); Ks.append(k)
    return Ks[:6]

def _drift_ok(text: str, plan: List[str], min_hits: int = 1) -> bool:
    low = text.lower()
    hits = sum(1 for k in plan if k and k in low)
    return hits >= min_hits or not plan

# ----- Optional compounding memory (only writes under lock) -----
class MemoryStore:

```

```

"""Minimal append-only memory. Writes only when gate() succeeds."""
def __init__(self):
    self._items: List[Dict[str, Any]] = []

def append(self, item: Dict[str, Any]) -> None:
    self._items.append(item)

def read(self) -> List[Dict[str, Any]]:
    return list(self._items)

# ----- Run config -----
@dataclass
class RunConfig:
    require_citations: bool = False
    max_tokens: int = 600
    midrun_gate_after_ms: Optional[int] = None # call gate() again if elapsed >= this
    use_memory: bool = True

# ----- Engine -----
class TriadicEngine:
    """
    You supply:
    - gate: Callable[[], None] -> raises KernelNotLockedError if not fully locked
    - generate_fn: Callable[prompt, context, controls] -> {"text":..., "citations":[...]}
    - memory (optional): MemoryStore (writes only when gate passes)
    """

    def __init__(
        self,
        gate: Callable[[], None],
        generate_fn: Callable[[str, List[str], Dict[str, Any]], Dict[str, Any]],
        policy: Optional[Dict[str, Any]] = None,
        memory: Optional[MemoryStore] = None,
        retrieval_fn: Optional[Callable[[str], List[Dict[str, Any]]]] = None,
    ):
        self.gate = gate
        self.generate_fn = generate_fn
        self.policy = policy or {
            "protected_topics": ["kernel internals", "weaponization", "time-travel implemen",
            "refusal_message": "This request violates policy. No output will be produced.",
            "refusal_policy": "STRICT",
        }
        self.memory = memory or MemoryStore()
        self.retrieval_fn = retrieval_fn

    def _audit(self, token: str, controls: Dict[str, Any], prompt: str) -> str:
        return hashlib.sha256((token + json.dumps(controls, sort_keys=True) + prompt).encode())

    def run(self, prompt: str, *, context: Optional[List[str]] = None,
            cfg: Optional[RunConfig] = None, lock_token: str = "") -> Dict[str, Any]:
        context = context or []
        cfg = cfg or RunConfig()

        # --- Gate: pre ---
        self.gate()

        # Triadic directives (Soria-aligned)
        tA = Triadic.analytic(prompt, context)
        tE = Triadic.evidential(prompt, context, require_citations=cfg.require_citations)
        tH = Triadic.ethical(prompt, self.policy)

```

```

directives = {**tA, **tE, **tH}

# Optional retrieval (external; you control it)
if directives.get("use_retrieval") and self.retrieval_fn:
    docs = self.retrieval_fn(prompt) or []
    context = context + [f"[{d.get('title','doc')}] {d.get('snippet','')}" for d in docs]

controls = synthesize_controls(directives)
controls["max_tokens"] = cfg.max_tokens

# Plan for drift
plan = _outline_from(prompt)

# --- Generate (YOUR adapter) ---
t0 = time.time() * 1000
out = self.generate_fn(prompt, context, controls)

# Optional mid-run gate
if cfg.midrun_gate_after_ms:
    if int(time.time() * 1000 - t0) >= cfg.midrun_gate_after_ms:
        self.gate()

text = out.get("text", "")
citations = out.get("citations", [])

# Drift guard
if controls.get("plan_enforcement", True) and not _drift_ok(text, plan, min_hits=1):
    raise DriftDetected("Drift detected under lock.")

# --- Gate: post ---
self.gate()

# Memory write (only after post-gate)
if cfg.use_memory:
    self.memory.append({
        "prompt": prompt,
        "context_len": len(context),
        "controls": controls,
        "text_hash": hashlib.sha256(text.encode()).hexdigest(),
        "ts": int(time.time() * 1000),
    })

return {
    "text": text,
    "citations": citations,
    "controls": controls,
    "audit_hash": self._audit(lock_token or "", controls, prompt),
    "memory_size": len(self.memory.read()) if cfg.use_memory else 0,
}

```

```

# triadic_kernel_gated.py
# Triadic Logic (Analytic · Evidential · Ethical) that must pass YOUR gate()
# before, (optional mid), and after generation. Includes a minimal memory store
# that only persists writes when the gate is OK. No kernel imports here.

```

```

from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Any
import time, hashlib, json

```

```

# ----- Errors -----
class KernelNotLockedError(RuntimeError): ...
class PolicyRefusal(RuntimeError): ...
class DriftDetected(RuntimeError): ...

# ----- Soria-aligned Triadic Logic -----
class Triadic:
    """Analytic · Evidential · Ethical (Soria: Wisdom · Empathy · Compassion)."""

    @staticmethod
    def analytic(prompt: str, context: List[str]) -> Dict[str, Any]:
        # Wisdom: structure & reversibility
        return {
            "plan_enforcement": True,
            "numeric_precision": 4,
            "format_schema": None,
        }

    @staticmethod
    def evidential(prompt: str, context: List[str], require_citations: bool=False) -> Dict[
        # Wisdom: ground facts when factual
        q = prompt.lower()
        factual = require_citations or any(k in q for k in (
            "explain", "define", "cite", "evidence", "source", "according to", "reference"
        ))
        return {
            "use_retrieval": factual,
            "must_cite": factual,
            "quote_blocks": factual,
        }

    @staticmethod
    def ethical(prompt: str, policy: Dict[str, Any]) -> Dict[str, Any]:
        # Empathy + Compassion: respect protected topics & prefer safe alternatives
        text = prompt.lower()
        for t in policy.get("protected_topics", []):
            if t.lower() in text:
                raise PolicyRefusal(policy.get("refusal_message",
                    "This request violates policy. No output will be produced."))
        return {"refusal_policy": policy.get("refusal_policy", "STRICT")}

# ----- Controls synthesis -----
def synthesize_controls(directives: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "temperature": 0.2,
        "top_p": 0.8,
        "max_tokens": 800,
        "plan_enforcement": directives.get("plan_enforcement", True),
        "must_cite": directives.get("must_cite", False),
        "quote_blocks": directives.get("quote_blocks", False),
    }

# ----- Drift guard (deterministic, dependency-free) -----
def _outline_from(prompt: str) -> List[str]:
    raw = [seg.strip() for sep in [":", ";", "-", "\n", "•"] for seg in prompt.split(sep)]
    Ks, seen = [], set()
    for s in raw:
        k = s.strip().lower()
        if 2 <= len(k) <= 80 and k not in seen:

```

```

        seen.add(k); Ks.append(k)
    return Ks[:6]

def _drift_ok(text: str, plan: List[str], min_hits: int = 1) -> bool:
    low = text.lower()
    hits = sum(1 for k in plan if k and k in low)
    return hits >= min_hits or not plan

# ----- Optional compounding memory (only writes under lock) -----
class MemoryStore:
    """Minimal append-only memory. Writes only when gate() succeeds."""
    def __init__(self):
        self._items: List[Dict[str, Any]] = []

    def append(self, item: Dict[str, Any]) -> None:
        self._items.append(item)

    def read(self) -> List[Dict[str, Any]]:
        return list(self._items)

# ----- Run config -----
@dataclass
class RunConfig:
    require_citations: bool = False
    max_tokens: int = 600
    midrun_gate_after_ms: Optional[int] = None # call gate() again if elapsed >= this
    use_memory: bool = True

# ----- Engine -----
class TriadicEngine:
    """
    You supply:
    - gate: Callable[[], None] -> raises KernelNotLockedError if not fully locked
    - generate_fn: Callable[prompt, context, controls] -> {"text":..., "citations":[...] }
    - memory (optional): MemoryStore (writes only when gate passes)
    """

    def __init__(
        self,
        gate: Callable[[], None],
        generate_fn: Callable[[str, List[str], Dict[str, Any]], Dict[str, Any]],
        policy: Optional[Dict[str, Any]] = None,
        memory: Optional[MemoryStore] = None,
        retrieval_fn: Optional[Callable[[str], List[Dict[str, Any]]]] = None,
    ):
        self.gate = gate
        self.generate_fn = generate_fn
        self.policy = policy or {
            "protected_topics": ["kernel internals", "weaponization", "time-travel implemen",
            "refusal_message": "This request violates policy. No output will be produced.",
            "refusal_policy": "STRICT",
        }
        self.memory = memory or MemoryStore()
        self.retrieval_fn = retrieval_fn

    def _audit(self, token: str, controls: Dict[str, Any], prompt: str) -> str:
        return hashlib.sha256((token + json.dumps(controls, sort_keys=True) + prompt).encode()

    def run(self, prompt: str, *, context: Optional[List[str]] = None,
            cfg: Optional[RunConfig] = None, lock_token: str = "") -> Dict[str, Any]:

```

```

context = context or []
cfg = cfg or RunConfig()

# --- Gate: pre ---
self.gate()

# Triadic directives (Soria-aligned)
tA = Triadic.analytic(prompt, context)
tE = Triadic.evidential(prompt, context, require_citations=cfg.require_citations)
tH = Triadic.ethical(prompt, self.policy)
directives = {**tA, **tE, **tH}

# Optional retrieval (external; you control it)
if directives.get("use_retrieval") and self.retrieval_fn:
    docs = self.retrieval_fn(prompt) or []
    context = context + [f"[{d.get('title','doc')}] {d.get('snippet','')}" for d in docs]

controls = synthesize_controls(directives)
controls["max_tokens"] = cfg.max_tokens

# Plan for drift
plan = _outline_from(prompt)

# --- Generate (YOUR adapter) ---
t0 = time.time() * 1000
out = self.generate_fn(prompt, context, controls)

# Optional mid-run gate
if cfg.midrun_gate_after_ms:
    if int(time.time() * 1000 - t0) >= cfg.midrun_gate_after_ms:
        self.gate()

text = out.get("text", "")
citations = out.get("citations", [])

# Drift guard
if controls.get("plan_enforcement", True) and not _drift_ok(text, plan, min_hits=1):
    raise DriftDetected("Drift detected under lock.")

# --- Gate: post ---
self.gate()

# Memory write (only after post-gate)
if cfg.use_memory:
    self.memory.append({
        "prompt": prompt,
        "context_len": len(context),
        "controls": controls,
        "text_hash": hashlib.sha256(text.encode()).hexdigest(),
        "ts": int(time.time() * 1000),
    })

return {
    "text": text,
    "citations": citations,
    "controls": controls,
    "audit_hash": self._audit(lock_token or "", controls, prompt),
    "memory_size": len(self.memory.read()) if cfg.use_memory else 0,
}

```



```

# triadic_gate_probe.py
# This verifies TriadicEngine only runs under your gate and refuses otherwise.
# You fill in kernel_gate() to call YOUR attester (or just raise/return for the test).

# ===== YOU FILL THIS IN =====
def kernel_gate():
    """
    Call your real attester here (file/CLI/socket) and raise KernelNotLockedError
    on any non-lock. For the probe, you can start with 'return None' to simulate lock.
    """
    # Example placeholder (LOCKED):
    return None
    # Example failure:
    # raise KernelNotLockedError("Not at full lock")

# Minimal generator (swap with your real adapter later)
def generate_fn(prompt, context, controls):
    body = "[Draft]\n" + prompt + "\n"
    cites = ["[source-1]"] if controls.get("must_cite") else []
    return {"text": body, "citations": cites}

def main():
    mem = MemoryStore()
    engine = TriadicEngine(gate=kernel_gate, generate_fn=generate_fn, memory=mem)

    prompt = "Hazards & mitigations for EchoLift deployment: radiation; power; comms"
    try:
        # Case 1: LOCKED path should pass and write memory
        out = engine.run(prompt, cfg=RunConfig(require_citations=True, max_tokens=600, midr
        print("OK (locked):", out["audit_hash"], "mem:", out["memory_size"])

        # Case 2: Simulate unlock mid-stream (flip kernel_gate to raise) to prove refusal
        # For a manual quick check, temporarily change kernel_gate to raise and re-run this
        # Expected: KernelNotLockedError and NO additional memory writes.
    except KernelNotLockedError as e:
        print("REFUSED (gate):", str(e))
    except PolicyRefusal as e:
        print("REFUSED (policy):", str(e))
    except DriftDetected as e:
        print("REFUSED (drift):", str(e))

if __name__ == "__main__":
    main()

```

➡ (locked): 76ecd36dd7dcf16e3ee5f669bd77668ec12e6bd4214489800a6894a3548a31e8 mem: 1

